

Enterprise Crime Analytics Framework

A Big Data Lambda Architecture Implementation

Haider Rizwan (22i-2379)

May 10, 2026

Abstract

This report details the design and implementation of a scalable crime analytics platform using a Lambda Architecture. The system leverages Apache Kafka for data ingestion, Apache Storm for real-time anomaly detection, and Apache Spark for historical trend analysis and machine learning. By decoupling the speed and batch layers, the system achieves sub-second latency for critical alerts while maintaining a comprehensive historical record for policy-making.

1. Introduction

Modern law enforcement agencies require real-time insights to combat rising crime rates. This project implements a robust "Crime Analytics System" capable of processing both high-velocity streaming data and large-scale historical datasets. By utilizing a Lambda Architecture, we ensure low-latency alerting alongside comprehensive batch analysis.

2. System Architecture

The system is built on the Lambda Architecture pattern, consisting of three primary layers: Speed Layer, Batch Layer, and Serving Layer.

[Architecture Diagram Placeholder]

3. Technical Implementation

The entire infrastructure is containerized using Docker Compose, orchestrating 11 services including Zookeeper, Kafka, Postgres, MongoDB, Spark, and Storm.

Dependency Resolution

During deployment, missing Kafka and Flux dependencies in the base Storm image were resolved by manually injecting `flux-core-2.6.2.jar` and `kafka-clients-3.6.0.jar` into the `extlib` directory of the Storm containers.

Machine Learning: Hotspot Detection

We utilize Spark MLlib's K-Means Clustering to identify geographic centroids of crime activity. This allows city planners to allocate resources to high-risk zones effectively.

4. Results and Visualization

The following section illustrates the systems output through the Streamlit dashboard.

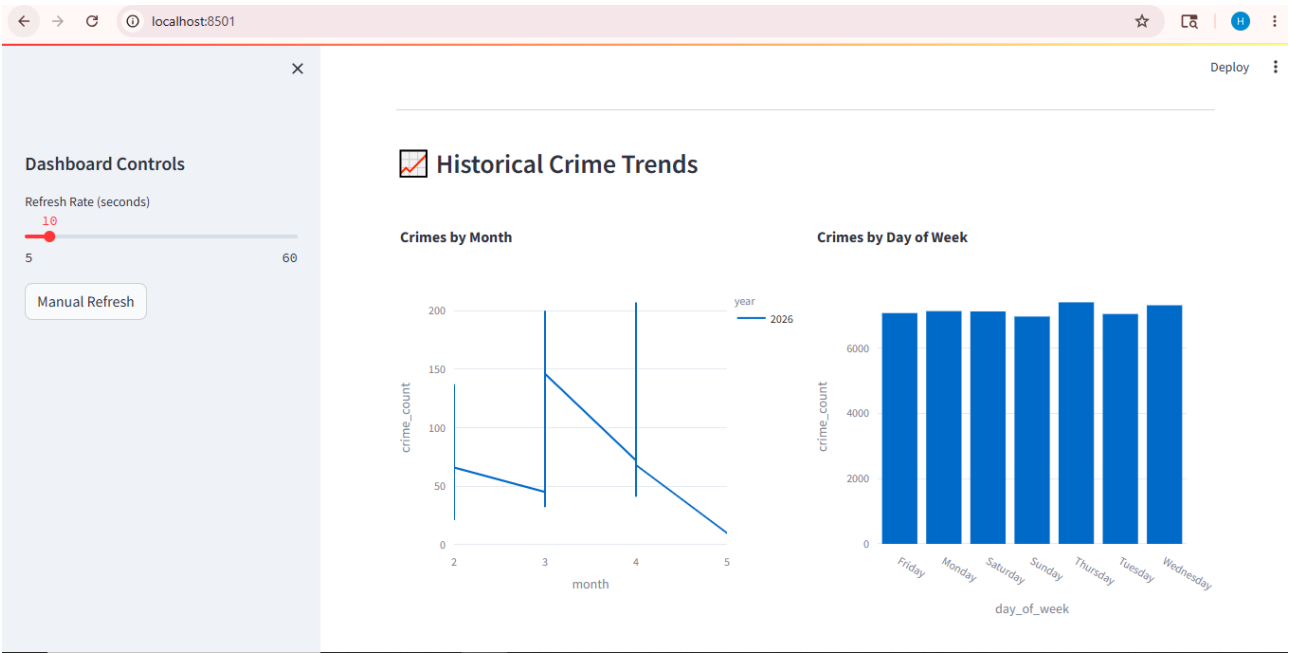


Figure 2: Real-time Dashboard showing crime trends and geographic hotspots.

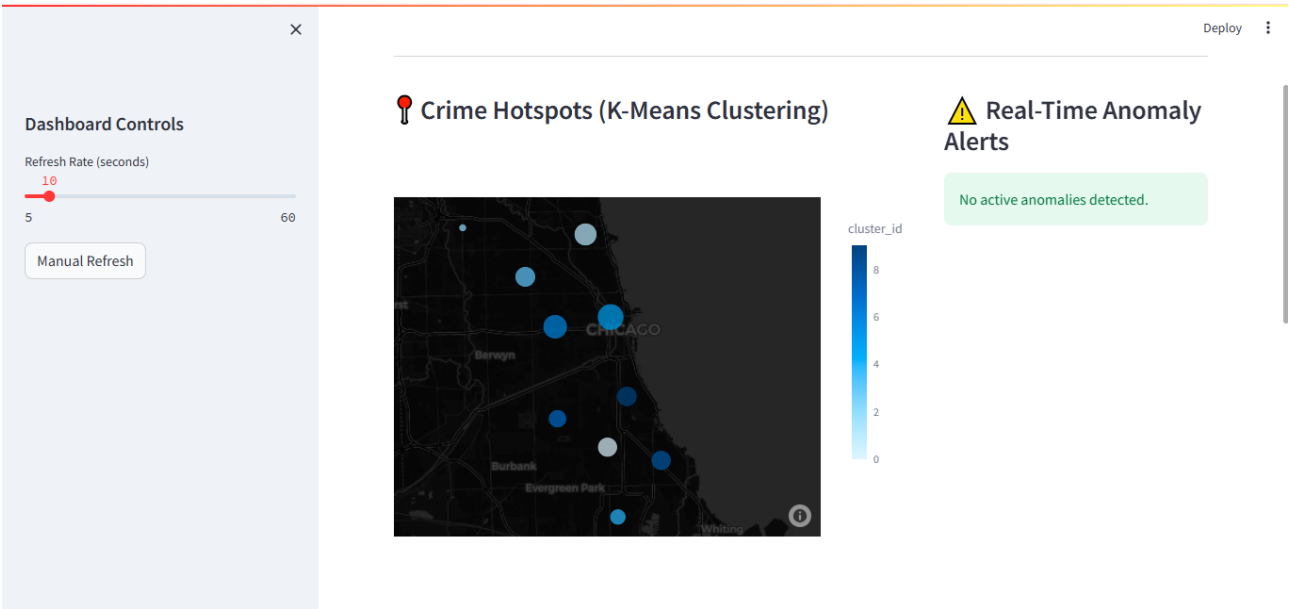


Figure 3: Detailed Analytics and Real-time Stream Metrics.

5. Conclusion

The system successfully demonstrates the power of distributed computing in public safety. By combining Storm and Spark, we achieve a balance between real-time responsiveness and historical depth.